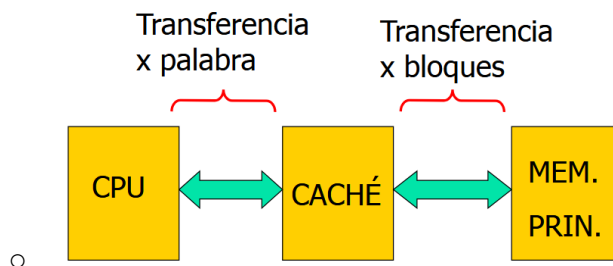


Arquitectura de Computadoras

- [Memoria](#)
- [Memoria caché](#)

Memoria

- Comunicación entre el CPU, la caché y la RAM:



-
- La jerarquía de memoria funciona por los principios de localidad espacial y temporal de los programas.
- La jerarquía debe cumplir con las siguientes propiedades:
 - Inclusión: Los datos en un nivel tienen que estar almacenados también en los niveles inferiores.
 - Coherencia: Las copias de los mismos datos en distintos niveles deben coincidir.
- Tipos de correspondencia
 - **Correspondencia directa**
 - Un bloque de la memoria tiene un lugar “reservado” en la caché, y solo puede copiarse allí. Distintos bloques de la memoria tienen el mismo lugar “reservado”.
 - **Correspondencia totalmente asociativa**
 - Un bloque de la memoria principal puede almacenarse en cualquier lugar libre de la caché
 - **Correspondencia asociativa por conjuntos**
 - Se divide la caché a la mitad. Un bloque de la memoria principal tiene un lugar “reservado” en cada mitad, y puede ir a cualquiera de esos 2 lugares.

Memoria caché

- Es una memoria pequeña y rápida, que copia aquellas celdas de la memoria principal con una alta probabilidad de ser necesitadas por el CPU (se tienen en cuenta los principios de localidad de los programas).
- Está formada por L1 y L2. L1 es la “caché de la caché”, es decir, recibe bloques de L2, y es aún más pequeña y rápida.
- Conceptos
 - **Acierto/hit:** La CPU busca un dato y lo encuentra en la caché
 - **Fallo/miss:** La CPU busca un dato y no lo encuentra en la caché
 - En ese caso, un bloque que contiene el dato buscado se copia a la caché
 - El tiempo necesario para servir un fallo depende de los siguientes factores:
 - **Latencia:** es el tiempo necesario para realizar un acceso a la memoria principal
 - **Ancho de banda:** es la cantidad de información que se puede transferir a/desde la memoria principal por unidad de tiempo.
- **Caché con correspondencia directa:**
 - ¿Cómo se determina qué lugar corresponde a un bloque N de la memoria?
 - $N \bmod \text{cantLineasDeCache} = \text{línea de caché que corresponde al bloque N.}$
 - **Índice:** porción del número de bloque que corresponde al número de la línea de caché en la que se almacenará (resultado de la cuenta anterior). No hace falta almacenarlo, porque se puede inferir de la posición de la caché en la que está guardado el bloque.
 - **BO:** Diferencia a cada celda que compone a un bloque de la caché. No se almacena porque dentro de la caché, las celdas de los bloques están ordenados (de la dirección más baja a la más alta), por lo que la CPU siempre sabe a cuál celda tiene que acceder.
 - Una vez que se encuentra la línea de caché en la que almacenar el bloque, ¿qué se almacena en dicha línea?
 - Componentes

- **Etiqueta:** porción restante del número de bloque (diferencia a los distintos bloques cuyo número de línea de caché es igual).
 - Contenido de cada celda del bloque, que se copia a la caché.
 - Ventajas
 - Simple y poco costosa
 - Es fácil de encontrar un dato en la caché (al tener una posición “reservada”, el CPU sabe dónde buscarlo)
 - Desventajas
 - A la hora de guardar un bloque, la caché puede estar libre excepto por una única línea, pero si el bloque que quiero guardar corresponde a la línea ya ocupada, habrá que sobrescribirlo.
 - Cuando el CPU intenta acceder frecuentemente a dos bloques cuyo número de línea de caché coincide, las pérdidas de tiempo (fallos) son altas.
- **Caché con correspondencia totalmente asociativa:**
 - Un bloque puede ir a cualquier lugar libre de la caché
 - ¿Qué se almacena en cada línea de caché?
 - **Etiqueta:** Es el número de bloque en su totalidad
 - **BO** funciona igual en todas las correspondencias.
 - Contenido de cada celda del bloque, que se copia a la caché.
 - Ventajas
 - Siempre que tenga un espacio libre en la caché, voy a poder guardar un bloque ahí.
 - La etiqueta identifica unívocamente un bloque de memoria.
 - Desventajas
 - Como un bloque puede estar en cualquier lado, el CPU tiene que recorrer todas las líneas de caché hasta que encuentra la que quiere. Un bloque es difícil de encontrar en la caché.
- **Caché con correspondencia asociativa por conjuntos:**
 - **Índice:** porción del número de bloque que corresponde al número de la línea de caché en la que se puede almacenar (resultado de la cuenta anterior). Es más chico que en la correspondencia directa, porque ahora hay una menor cantidad de líneas de caché (ya que

cada bloque se puede almacenar en 2 líneas de caché distintas, pero ambas con el mismo número).

- **BO** funciona igual en todas las correspondencias.
- ¿Qué se almacena en cada línea de caché?
 - **Etiqueta:** Porción restante del número de bloque (diferencia a los distintos bloques cuyo número de línea de caché es igual).
Es más grande que en la correspondencia directa, porque ahora hay más bloques correspondientes a cada número de caché.
 - Contenido de cada celda del bloque, que se copia a la caché.
- Ventajas y desventajas
 - Comparte lo mejor de las otras dos correspondencias.
- Política de reemplazos
 - ¿Qué pasa si quiero guardar algo en la caché, pero el lugar en el que quiero guardarlo ya está ocupado?
 - Correspondencia directa
 - Se sobrescribe el dato en la ubicación que corresponde.
 - Correspondencia totalmente asociativa
 - Se debe sobrescribir una línea de la caché, y hay distintos algoritmos para decidir cuál:
 - **LRU** (least recently used)
 - Hay una lista con las líneas ordenadas. Primero en la lista está al que se accedió por última vez, y último está el que hace más tiempo que no se accede. El último se sobrescribe
 - Requiere controles de tiempos
 - **LFU** (least frequently used)
 - Se elimina la celda que menos accesos totales tuvo.
 - Requiere controles de uso.
 - **FIFO** (first in first out)
 - Requiere controles de acceso.
 - **Random**
 - Correspondencia asociativa por conjuntos
 - Si ambas líneas de caché que corresponden a un bloque están ocupadas, se usa uno de los algoritmos mencionados previamente para decidir cuál de los 2 sobrescribir.

- Política de escritura
 - Cuando el CPU tiene que cambiar un dato de la memoria, lo modifica en la caché. ¿Qué pasa con ese dato en la memoria principal?
 - En acierto (el dato que quiero cambiar está en la caché)
 - **Write-through (escritura inmediata):** se actualizan simultáneamente la caché y la memoria principal.
 - Hay más tráfico entre la caché y la memoria principal.
 - La información de la caché y la memoria principal siempre es correcta.
 - **Write-back (post-escritura):** la información solo se actualiza en la caché, y se marca con un bit de “sucio”. Una vez que el bloque de la caché es sobreescrito, la CPU sabe que tiene que actualizar la memoria principal gracias a dicho bit.
 - Se reduce el tráfico entre la caché y la memoria principal.
 - Hay momentos en los que la memoria principal tiene información errónea.
 - En fallo (el dato que quiero cambiar no está en la caché)
 - **No-write allocate:** El bloque no se lleva a la memoria caché. Se modifica directamente en la memoria principal.
 - Habitual con write-through.
 - **Write-allocate:** La información se lleva de la memoria principal a la caché. Se modifica en la caché.
 - Habitual con write-back.